

Agile Project Plan and Sprint Board: Complete Release Dashboard

Project: AI Placement Management System

Welcome to the Agile Sprint Board for the college project AI Placement Management System. This dashboard serves as the single source of truth for the entire development and release cycle.

1. Sprint Overview

Sprint Name: Complete AI Placement Management System Sprint

Sprint Period: 13 August 2026 – 25 August 2026

Sprint Goal

"Develop, integrate, test, and demonstrate the complete AI Placement Management System within the period from 13 August 2026 to 25 August 2026, implementing the Student + AI, Company + Recruitment, and Admin + Analytics modules as one integrated end-to-end placement workflow."

Team Structure & Coordinated Pairs

The development team consists of 7 members working in 3 coordinated pairs:

Pair 1 — Student + AI

- Anshika Srivastava (IN26014085)
- Sanskriti Singh (IN26014091)
- Focus: Student registration, profiles, resume upload, FastAPI AI matching, compatibility scores, and applications.

Pair 2 — Company + Recruitment

- Saaksham Kindra (IN26013408)
- Punit Kumar (IN26012858)
- Sudip Aryan (IN26012640)
- Focus: Recruiter authentication, company profiles, job-drive wizard, candidate shortlisting, interview scheduler, and status pipeline.

Pair 3 — Admin + Analytics

- Shriya Punnal (IN26014131)
- Urmi Barman (IN26012701)
- Focus: Admin authentication, user records lists, job drive approvals, application monitoring, and placement dashboard analytics.

A fully operational, integrated product consisting of:

1. A consolidated SQL Server database structure populated via EF Core migrations.
2. Working Identity authentication with role-based authorization rules.
3. A shared Razor-based MVC UI design system with role-specific navigation menus.
4. Fully documented REST API contracts accessible via Swagger UI.
5. Functional student profiles, company profiles, and Job Drive publishing wizard.
6. A Python FastAPI AI service running NLP-based resume parsing, skill extraction, compatibility scoring, and skill-gap mappings.
7. An integrated recruitment status pipeline (Applied, Shortlisted, Assessment, Interview, Selected/Rejected).

8. Admin panels with interactive charts showing placement statistics and downloadable reports.

2. Project Requirements

The functional requirements are converted into Agile user stories below:

Shared Foundation (SF)

- SF-US-01: As a developer, I want to set up the ASP.NET Core solution structure, so that the team can build in a decoupled core layers pattern. [Must Have]
- SF-US-02: As a developer, I want to configure Entity Framework Core and run migrations, so that schemas are automatically created. [Must Have]
- SF-US-03: As a user, I want to register and log in according to my role, so that my security permissions are verified. [Must Have]
- SF-US-04: As a developer, I want to define standard request wrappers and enable Swagger UI, so that API testing is simplified. [Must Have]
- SF-US-05: As a user, I want to access a unified navigation template with a common CSS style guide, so that the interface is responsive. [Must Have]
- SF-US-06: As a developer, I want to establish git policies and code-review checks, so that the code remains buildable. [Must Have]

Student + AI (ST)

- ST-US-01: As a Student, I want to create an account, so that I can access my dashboard. [Must Have]
- ST-US-02: As a Student, I want to manage my CGPA, branch, and project details, so that my profile records are updated. [Must Have]
- ST-US-03: As a Student, I want to upload my resume, so that recruiters can download it. [Must Have]
- ST-US-04: As a Student, I want the system to extract my skills automatically using NLP, so that my profile is populated. [Must Have]
- ST-US-05: As a Student, I want to browse job openings and verify my eligibility against requirements, so that I can apply. [Must Have]
- ST-US-06: As a Student, I want to see a compatibility match score and skill gaps, so that I can prepare for matching roles. [Must Have]
- ST-US-07: As a Student, I want to submit applications and track their pipeline status, so that I can monitor my hiring stages. [Must Have]

Company + Recruitment (CO)

- CO-US-01: As a Recruiter, I want to update our company description, website, and details, so that candidates are informed. [Must Have]
- CO-US-02: As a Recruiter, I want to configure job requirements (CGPA, backlogs, branch), so that eligibility checks can occur. [Must Have]
- CO-US-03: As a Recruiter, I want to view candidates who applied and see their AI match scores, so that shortlisting is streamlined. [Must Have]
- CO-US-04: As a Recruiter, I want to move candidates through recruitment stages (Shortlist, Assessment, Interview), so that stages are tracked. [Must Have]
- CO-US-05: As a Recruiter, I want to schedule technical/HR rounds and log final results, so that selections are finalized. [Must Have]

Admin + Analytics (AD)

- AD-US-01: As a Placement Admin, I want to log in, so that I can access restricted admin dashboard systems. [Must Have]
- AD-US-02: As a Placement Admin, I want to view and manage registered student and recruiter records, so that database entries are correct. [Must Have]
- AD-US-03: As a Placement Admin, I want to review and approve job drives, so that they become visible to eligible students. [Must Have]
- AD-US-04: As a Placement Admin, I want to track applications and final selections, so that the placement database is updated. [Must Have]
- AD-US-05: As a Placement Admin, I want to view placement percentage, average salary packages, and branch statistics, so that reports can be generated. [Must Have]

3. Prioritization & Integration Workflow Plan

Because the entire project must be completed and demonstrated by August 25, 2026, all core features required to establish the end-to-end placement workflow are classified as Must Have. The core success scenario follows this sequence:

Student Registration -> Student Profile -> Resume Upload -> AI Resume Analysis -> Company Creates Job Drive -> Admin Approves Job Drive -> Eligibility Check -> AI Match Score -> Skill Gap -> Student Applies -> Recruiter Reviews -> Shortlist -> Assessment -> Technical Interview -> HR Interview -> Selection/Rejection -> Placement Result -> Admin Dashboard / Analytics

4. Sprint Workplan

ID	User Story	Task	Module	Priority	Story Points	Assignee	Dependency	Status
TSK-01	SF-US-02	Create ER database design and tables schema mapping.	Shared	Must Have	5	Shared Work	None	To Do
TSK-02	SF-US-06	Set up Git repository and create development branches.	Shared	Must Have	2	Shared Work	None	To Do
TSK-03	SF-US-01	Setup VS C# clean solution architectural layers.	Shared	Must Have	3	Shared Work	TSK-02	To Do
TSK-04	SF-US-02	Configure EF Core DbContext & SQL Server connection.	Shared	Must Have	3	Shared Work	TSK-01, TSK-03	To Do
TSK-05	SF-US-03	Setup Identity Auth & database roles configuration.	Shared	Must Have	8	Shared Work	TSK-04	To Do
TSK-06	SF-US-04	Standardize API response models and add Swagger UI.	Shared	Must Have	3	Shared Work	TSK-03	To Do
TSK-07	SF-US-05	Build MVC common Razor layout view & CSS styles.	Shared	Must Have	5	Shared Work	TSK-03	To Do
TSK-08	ST-US-01	Build Student Registration/Login MVC views & APIs.	Student	Must Have	3	Pair 1	TSK-05, TSK-07	To Do
TSK-09	ST-US-02	Build Student Profile fields	Student	Must Have	5	Pair 1	TSK-04, TSK-07	To Do

		(CGPA, Branch, Backlogs).						
TSK-10	ST-US-03	Build Student Resume upload views & file saving APIs.	Student	Must Have	3	Pair 1	TSK-09	To Do
TSK-11	CO-US-01	Build Recruiter Registration/Login MVC views & APIs.	Company	Must Have	3	Pair 2	TSK-05, TSK-07	To Do
TSK-12	CO-US-01	Build Company Profile setup views & update APIs.	Company	Must Have	3	Pair 2	TSK-04, TSK-07	To Do
TSK-13	CO-US-02	Build Job Drive creation and criteria configuration forms.	Company	Must Have	5	Pair 2	TSK-12	To Do
TSK-14	AD-US-01	Build Admin Login view and role authorization check.	Admin	Must Have	2	Pair 3	TSK-05, TSK-07	To Do
TSK-15	AD-US-03	Build Admin Job Drive approval and rejection workflows.	Admin	Must Have	3	Pair 3	TSK-13, TSK-14	To Do
TSK-16	ST-US-05	Build Student Job browsing and eligibility checking APIs.	Student	Must Have	5	Pair 1	TSK-09, TSK-15	To Do
TSK-17	ST-US-04	Build Python FastAPI NLP skill extraction microservice.	Student	Must Have	8	Pair 1	TSK-10	To Do
TSK-18	ST-US-06	Build AI compatibility scoring & skill-gap algorithms.	Student	Must Have	5	Pair 1	TSK-17, TSK-13	To Do
TSK-19	ST-US-07	Build Student Application views and submission routes.	Student	Must Have	3	Pair 1	TSK-16, TSK-18	To Do
TSK-20	CO-US-03	Build Recruiter applicant list & match score reviews.	Company	Must Have	5	Pair 2	TSK-19	To Do
TSK-21	CO-US-04	Build Recruiter status updates (Applied to Shortlisted).	Company	Must Have	3	Pair 2	TSK-20	To Do
TSK-22	CO-US-05	Build Recruiter interview scheduling & logging views.	Company	Must Have	5	Pair 2	TSK-21	To Do
TSK-23	AD-US-04	Build Admin application and placement outcomes panels.	Admin	Must Have	3	Pair 3	TSK-22	To Do
TSK-24	AD-US-05	Build Admin dashboard metrics & branch analytics charts.	Admin	Must Have	5	Pair 3	TSK-23	To Do
TSK-25	AD-US-05	Build Admin downloadable placement reports queries.	Admin	Must Have	3	Pair 3	TSK-24	To Do
TSK-26	Integration	Integrate ASP.NET Core backend with FastAPI	Shared	Must Have	8	Shared Work	TSK-17, TSK-18	To Do

		service.						
TSK-27	Testing	Perform unit tests, E2E workflow, and security checks.	Shared	Must Have	5	Shared Work	All Tasks	To Do
TSK-28	Sign-Off	Scrub final bugs, write README, prepare system demo.	Shared	Must Have	3	Shared Work	TSK-27	To Do

5. Task Breakdown

Shared Foundation (SF)

- Setup ASP.NET Core Clean Architecture layered reference projects structure.
- Initialize SQL Server and map database tables using EF Core.
- Integrate Identity authentication and configure authorization filters.
- Develop standard JSON response wrappers and Swagger UI configurations.
- Design MVC shared Razor layout views, dynamic sidebar menus, and site styles.

Student + AI (ST)

- Design StudentProfile DB schemas and Razor profile creation forms.
- Configure resume file upload models validating PDF file extensions.
- Build Python FastAPI microservice utilizing NLP models for resume skill extraction.
- Implement matching algorithms to calculate candidate compatibility scores.
- Build student job browsing list views mapping eligibility status.
- Implement job application submission controllers.

Company + Recruitment (CO)

- Design CompanyProfile details edit forms.
- Design recruiter Job Drive creation wizards mapping criteria fields.
- Build recruiter applicant overview tables displaying AI compatibility scores.
- Build applicant recruitment pipeline advancement triggers.
- Design recruiter interview scheduling calendar views.
- Build recruiter select/reject buttons logging placement outcomes.

Admin + Analytics (AD)

- Design Admin control panel tables listing students and companies.
- Build Admin review dashboards showing pending Job Drive approval forms.
- Develop Admin DB query counters tracking registered statistics.
- Build chart integration displaying branch placement percentages.
- Develop reporting scripts to query and export placement spreadsheets.

6. Dependency Management

The project tasks are sequence-chained as follows:

ER Design (TSK-01) -> Database Schema -> EF Core (TSK-04) -> API Models / DTOs -> Authentication & Authorization (TSK-05) -> MVC Layout Foundation (TSK-07) -> Student/Company/Admin Features -> AI FastAPI Integration (TSK-17, TSK-26) -> Application & Recruitment Workflow (TSK-19, TSK-22) -> Placement & Analytics (TSK-24) -> End-to-End Integration -> Testing (TSK-27) -> Bug Fixing -> Final Demonstration (TSK-28)

7. 10-Day Development Timeline

The release sprint runs from August 16 to August 25, 2026:

- 13 August: Sprint planning and ER design. Set up Git repository and solution reference structure.
- 17 August: Initialize SQL Server database and run EF migrations. Setup Identity authentication and seed roles.
- 18 August: Build MVC layout views and navigation menus. Create registration/login views for all roles.
- 19 August: Create Student and Company profiles edit views. Build recruiter Job Drive publishing form.
- 20 August: Build Python FastAPI microservice. Extract skills and compute compatibility scores.
- 21 August: Build student job browsing, eligibility check, application submission, and recruiter applicant lists.
- 22 August: Build recruiter interview scheduling, admin outcomes tracking, and analytics dashboards.
- 23 August: Integrate ASP.NET Core with FastAPI. Resolve merge conflicts and run E2E data tests.
- 24 August: Execute unit, integration, and role security tests. Freeze code and debug layouts.
- 25 August: Verify Swagger docs and seed data. Present final E2E demonstration to the project guide.

8. Daily Task Assignment

Date	Pair 1 — Student + AI	Pair 2 — Company + Recruitment	Pair 3 — Admin + Analytics	Shared Work
13 Aug	Setup Git branch guidelines	Setup Git branch guidelines	Setup Git branch guidelines	ER Design (TSK-01), solution setup (TSK-03)
14-15 Aug	Support DB migration	Support DB migration	Support DB migration	EF Core DB & Identity setups (TSK-04, TSK-05)
16-17 Aug	Create Student registration UI	Create Recruiter registration UI	Create Admin Login UI	Common MVC layout design (TSK-07, TSK-06)
18-19 Aug	Build Student Profile (TSK-09, TSK-10)	Build Company Profile (TSK-12, TSK-13)	Build Admin dashboard layout (TSK-15)	Review database records seeds
20 Aug	Build FastAPI NLP (TSK-17, TSK-18)	Support API schema setup	Configure admin student list view	Review AI integration contracts
21 Aug	Build Job Apply routes (TSK-16, TSK-19)	Build Recruiter status views (TSK-20, TSK-21)	Configure admin company list view	E2E API schema check
22 Aug	Verify application logs	Build Recruiter Interview forms (TSK-22)	Build Admin analytics (TSK-23, TSK-24, TSK-25)	Run data logs check
23 Aug	FastAPI Integration	Recruiter pipeline check	Admin panel verify	Integrate API with FastAPI & DB (TSK-26)
24 Aug	Execute unit/API tests	Execute UI forms tests	Execute security tests	E2E verification & bug fixes (TSK-27)
25 Aug	Prepare final demo	Prepare final demo	Prepare final demo	Final project signoff & demonstration (TSK-28)

--- | :--- | :--- | :--- | :--- |

16 Aug	Review requirements	Review requirements	Review requirements	ER Design (TSK-01), solution setup (TSK-03)
17 Aug	Support DB migration	Support DB migration	Support DB migration	EF Core DB & Identity setups (TSK-04, TSK-05)
18 Aug	Build Student login (TSK-08)	Build Recruiter login (TSK-11)	Build Admin Login UI (TSK-14)	Common MVC layout design (TSK-07, TSK-06)
19 Aug	Build Student Profile (TSK-09, TSK-10)	Build Company Profile (TSK-12, TSK-13)	Build Admin dashboard layout (TSK-15)	Review database records seeds
20 Aug	Build FastAPI NLP (TSK-	Support API schema	Configure admin	Review AI integration

	17, TSK-18)	setup	student list view	contracts
21 Aug	Build Job Apply routes (TSK-16, TSK-19)	Build Recruiter status views (TSK-20, TSK-21)	Configure admin company list view	E2E API schema check
22 Aug	Verify application logs	Build Recruiter Interview forms (TSK-22)	Build Admin analytics (TSK-23, TSK-24, TSK-25)	Run data logs check
23 Aug	FastAPI Integration	Recruiter pipeline check	Admin panel verify	Integrate API with FastAPI & DB (TSK-26)
24 Aug	Execute unit/API tests	Execute UI forms tests	Execute security tests	E2E verification & bug fixes (TSK-27)
25 Aug	Prepare final demo	Prepare final demo	Prepare final demo	Final project signoff & demonstration (TSK-28)

9. Acceptance Criteria

- Registration & Login: Student, recruiter, and admin log in successfully. User is redirected to corresponding dashboard path: /Student/Dashboard, /Company/Dashboard, /Admin/Dashboard.
- Access Restrictions: Manually navigating to routes outside authorized roles returns an HTTP 403 Forbidden page.
- Student Profiles: CGPA is validated to values between 0.00 and 10.00. Resume upload rejects non-PDF files.
- AI matching: Python FastAPI NLP model extracts skills and computes candidate matching compatibility scores.
- Recruiter Job drives: Recruiter publishes drives specifying criteria. Recruiter applicant list displays candidate profiles with AI match scores.
- Admin dashboard: Admin Approve/Reject buttons change job drive visibility. Admin charts display correct placement percentages.

10. Definition of Done (DoD)

A task is declared Done only when:

- Code complies with clean architecture layers.
- DB migrations are run and updated on SQL Server.
- REST APIs are verified using Swagger.
- Auth filters block unauthorized access.
- Code passes peer review and is merged to dev branch.
- End-to-end placement scenario executes without errors.

11. Sprint Capacity Planning

- Pair 1 Capacity: 60 available hours (10 days × 6 hours/day) | SP Budget: 30 SP
- Pair 2 Capacity: 60 available hours (10 days × 6 hours/day) | SP Budget: 30 SP
- Pair 3 Capacity: 60 available hours (10 days × 6 hours/day) | SP Budget: 30 SP
- Shared Capacity: Coordination, database schema design, and E2E testing | SP Budget: 10 SP
- Total Capacity: 360 Hours | Total Planned SP: 100 SP
- Remaining Contingency Capacity: 20 SP (16.7% reserved for bug scrubbing, integration, and final demo verification).

12. Risk Management Matrix

Identified Risk	Prob	Impact	Mitigation Action Plan
Late database schema	Medium	High	Freeze ER diagram on Day 1.

changes			Run migrations only through dev branch updates.
API schema incompatibility	High	High	Establish API request/response classes on Day 1. Mock responses before coding.
Identity auth filters failures	Medium	High	Setup base authorization filters on Day 2. Audit access rules daily.
FastAPI AI integration delays	High	High	Use mock data endpoints in Web API on Day 4. Run early cross-service checks.
Git merge conflicts	High	Medium	Enforce daily pull request reviews. Never commit to dev without peer checks.
Insufficient time (Aug 25 deadline)	Medium	High	Strictly limit scope to requirements. Use 16.7% contingency budget.

13. Daily Stand-Up & Blocker Tracking

Daily stand-ups occur at 9:00 AM. Developers report:

9. What did I complete yesterday?
10. What will I complete today?
11. What blockers am I currently facing?

Blocker Escalation Framework:

- Log blocker in shared tracking sheet.
- Pair 3 assigns corresponding developers to address blockers.
- If unresolved within 4 hours, a review meeting is called to adjust technical paths.

14. Sprint Review Checklist (25 August Demo)

The final demonstration on August 25, 2026, must display:

- Student account registration, login, and dashboard redirect.
- Student profile data entry, CGPA verification, and PDF resume upload.
- Recruiter account registration, login, and profile management.
- Recruiter Job Drive setup with eligibility thresholds.
- Admin login, student/company list, and Job Drive Approve/Reject controls.
- Student job browsing displaying approved opening, eligibility, match score, and skill gaps.
- Student application submission.
- Recruiter applicant overview displaying candidate scores.
- Recruiter workflow advancing candidate status (Shortlist, Assessment, Interview).
- Recruiter interview scheduler and result logs.
- Placement outcome persistence and admin dashboard stats refresh.
- Swagger UI API endpoints validation and xUnit green test explorer run.

15. Final End-to-End Success Scenario

Student registers -> Completes academic profile (8.5 CGPA, IT Branch) -> Uploads resume -> AI service parses resume and extracts skills -> Company recruiter registers -> Recruiter creates Job Drive (Min CGPA: 8.0, Allowed branch: IT) -> Admin logs in -> Admin approves Job Drive -> Student logs in -> System validates student eligibility -> AI calculates match score and skill gaps -> Student submits application -> Recruiter reviews application

alongside AI compatibility details -> Recruiter shortlists student -> Recruiter schedules interview rounds -> Recruiter logs final selection -> Placement Result table records starting package -> Admin Dashboard displays updated placement counts, branch percentage charts, and packages.

16. Final Deliverables Checklist

- Requirement Analysis Document: Checked / Complete
- Database ER Diagram & Schema: Checked / Complete
- ASP.NET Core MVC Web Solution: Checked / Complete
- FastAPI Python Microservice: Checked / Complete
- Role-based Security Identity rules: Checked / Complete
- REST APIs with Swagger XML comments: Checked / Complete
- E2E Integration Verification proof: Checked / Complete
- Unit/Integration test explorer logs: Checked / Complete
- System documentation & README: Checked / Complete

17. Final Sprint Status Workplan Board

- Backlog: TSK-25 (Downloadable Reports), TSK-28 (Final Presentation Signoff)
- To Do: TSK-23 (Admin Outcomes panel), TSK-24 (Admin dashboard metrics charts)
- In Progress: TSK-18 (AI scoring algorithms), TSK-19 (Student job applications routes), TSK-22 (Recruiter interview scheduler)
- Code Review: TSK-16 (Student Job browsing views), TSK-20 (Recruiter applicant listing)
- Testing: TSK-08 (Student login views), TSK-11 (Recruiter login views), TSK-17 (FastAPI NLP microservice)
- Done: TSK-01 (ER Schema), TSK-02 (Git setup), TSK-03 (Solution setup), TSK-04 (EF Core Database connection), TSK-05 (Identity authentication setup), TSK-06 (API Swagger wrappers), TSK-07 (MVC main layout UI), TSK-09 (Student profile configurations), TSK-10 (Resume upload endpoints), TSK-12 (Company profile views), TSK-13 (Job drive criteria setup), TSK-14 (Admin login validation)

18. System Completion Rules

The project is declared complete on August 25, 2026, only if:

- All three functional modules are integrated and running.
- Authentication and role-based security filters are active.
- MVC, API, SQL Server database, and FastAPI microservice communicate without errors.
- No critical bugs or formatting breaks remain in the UI.
- Final E2E demonstration scenario executes successfully.